

# MOTIONS

---

**w** / **b**

## HOP A WORD FORWARD / BACK

e = end of word · stop mashing llll

**f** *char*

## JUMP TO THAT CHAR ON THE LINE

; = next match , = back · t stops just before it

**0** / **\$**

## START / END OF LINE

^ = first non-blank character

**{** / **}**

## BLOCK UP / BLOCK DOWN

jumps by paragraph — fast vertical travel

**CTRL** **d** / **u**

## HALF-PAGE DOWN / UP

cursor stays centered-ish — smooth scroll is on

**12** **j**

## LEAP 12 LINES DOWN

the relative numbers in the gutter ARE the count — read, type, land

**gg** / **G**

## TOP / BOTTOM OF FILE

**\***

## SEARCH WORD UNDER CURSOR

n = next hit N = previous · # searches backwards

**/** *text*

## SEARCH ANYTHING

n / N to walk hits · ESC clears the highlight

# LSP NAVIGATION

---

**g r d**

## GO TO DEFINITION

**CTRL t**

## JUMP BACK

CTRL-o = back anywhere   CTRL-i = forward again

**g r r**

## EVERY REFERENCE

CTRL-ENTER in the list = open in vertical split

**g r i**

## GO TO IMPLEMENTATION

g r t = type definition

**g 0**

## EVERY SYMBOL IN THIS FILE

jump straight to any function — no scrolling

**g W**

## EVERY SYMBOL IN THE PROJECT

**K**

## DOCS UNDER CURSOR

**zM / zR**

## FOLD ALL / UNFOLD ALL

fold first, skim the skeleton · za = toggle one   zp = peek inside

# FILE & SEARCH PICKERS

---

**SPC** **F**

**FIND FILE BY NAME**

**SPC** **G**

**GREP THE WHOLE PROJECT**

live ripgrep — pattern is a regex

**SPC** **SPC**

**OPEN BUFFERS**

**SPC** **/**

**FUZZY SEARCH THIS FILE**

**SPC** **s** **w**

**GREP WORD UNDER CURSOR**

**SPC** **e**

**FILE TREE SIDEBAR**

q closes it · tree is for orienting — pickers are for going

**SPC** **s** **r**

**RESUME LAST PICKER**

SPC s . = recent files

**SPC** **s** **k**

**SEARCH EVERY KEYMAP**

forgot a binding? never leave the editor

**CTRL** **q**

**RESULTS → QUICKFIX LIST**

inside any picker · then :cnext or :cfdi to operate on all of them

# TEXT OBJECTS & EDITING

---

**c i "**

## CHANGE INSIDE QUOTES

works from anywhere on the line

**c i a**

## CHANGE THIS ARGUMENT

a = argument in any function call

**d a )**

## DELETE AROUND PARENS

i = inside a = around · also b " ' { [

**y a f**

## YANK FUNCTION CALL

c i n q = change inside NEXT quotes

**s a iw "**

## WRAP WORD IN QUOTES

visual: s a ) wraps the selection

**s r " '**

## SWAP " FOR '

s d " deletes the surround entirely

**⌘ /**

## TOGGLE COMMENT

normal, visual, and insert mode

**⌘ S**

## SAVE — ANY MODE

save also auto-formats

**.**

## REPEAT LAST EDIT

\* then n . n . = hand-guided refactor

# DIAGNOSTICS & FIXING

---

**gre** / **grE**

## NEXT / PREV DIAGNOSTIC

hop squiggle to squiggle

**SPC** **c** **a**

## CODE ACTION — THE FIX MENU

auto-import, quick-fix, refactor suggestions

**g** **r** **n**

## RENAME SYMBOL EVERYWHERE

semantic, cross-file — always try this before find & replace

**SPC** **s** **d**

## ALL PROJECT DIAGNOSTICS

the damage report after a big refactor

**SPC** **f**

## FORMAT NOW

js/ts formats only when an eslint config exists — that's by design

**SPC** **S** **r**

## SPECTRE: FIND & REPLACE

project-wide with per-match approval · **t f** = literal (non-regex) mode

**:cfd** **o %s/old/new/ge | update**

## EDIT EVERY QUICKFIX FILE

grep → CTRL-q → this = surgical mass edit

# GIT HUNKS & STAGING

---

**]c** / **[c**

## NEXT / PREV CHANGE

walk your uncommitted hunks

**SPC** **h** **s**

## STAGE THIS HUNK

works on a visual selection too · **SPC h S** stages the file

**SPC** **h** **r**

## DISCARD THIS HUNK

surgical undo — kill the debris, keep the commit clean

**SPC** **h** **p**

## PREVIEW HUNK DIFF

**SPC** **h** **b**

## BLAME THIS LINE

**SPC t b** = blame every line, inline, always on

**SPC** **g** **g**

## LAZYGIT

commit, push, branch, rebase — full TUI in a float

**SPC** **g** **e**

## GIT STATUS IN SIDEBAR

**SPC** **g** **o** **b**

## OPEN LINE ON GITHUB

visual-select first to link a range — instant share link

# DEBUGGING WITH DAP

---

**SPC** **b**

## TOGGLE BREAKPOINT

press, then PAUSE — typing another key makes it a buffer command

**SPC** **B**

## CONDITIONAL BREAKPOINT

prompts for the condition — break only when it matters

**F5**

## RUN / CONTINUE

**F2**

## STEP OVER

**F1**

## STEP INTO

**F3**

## STEP OUT

**F7**

## TOGGLE DEBUG UI

also shows the last session's output · UI auto-opens on start

# BUFFERS, SPLITS, TERMINAL

---

**CTRL** **,** **/** **.**

**PREV / NEXT BUFFER**

**CTRL** **p**

**PICK BUFFER BY LETTER**

letters appear on each tab — faster than cycling past 4 buffers

**SPC** **x**

**CLOSE BUFFER**

CTRL-c does it too

**SPC** **b** **o**

**CLOSE ALL OTHERS**

type it fast — pausing after SPC b drops a breakpoint

**SPC** **b** **P**

**PIN BUFFER**

then SPC b O closes everything except pinned

**CTRL** **h j k l**

**HOP BETWEEN SPLITS**

**CTRL** **/**

**TOGGLE TERMINAL**

session survives hiding — leave the test watcher running in it

**ESC** **ESC**

**LEAVE TERMINAL MODE**

back to normal mode so you can scroll and yank output



# OIL FILE MANAGER

---

-

## OPEN FOLDER AS A BUFFER

oil — the directory becomes editable text

*edit name* + :w

## RENAME

use real vim editing: visual block, :s, macros — mass rename ten files

*new line* + :w

## CREATE FILE

end the name with / to make a directory

dd + :w

## DELETE — GOES TO TRASH

not permanent, you configured that

dd ... p + :w

## MOVE FILE

delete the line here, paste it in another oil buffer, write

CTRL s

## OPEN IN SPLIT

CTRL-t = tab CTRL-p = preview

g.

## TOGGLE HIDDEN FILES

g?

## ALL OIL KEYS

SPC r p / SPC r P copy the path under the cursor